

UNIVERSIDADE FEDERAL DE ALAGOAS

CAMPUS A.C. SIMÕES

ENGENHARIA DE COMPUTAÇÃO

DAVI CELESTINO DOS SANTOS BARBOSA, 23210541

JOÃO VICTOR TENÓRIO VENANCIO, 23211191

OTAVIO JOSHUA COSTA BRANDAO MENEZES, 23211189

LARISSA FERREIRA DIAS DE SOUZA, 23211102

**RELATÓRIO TÉCNICO: SISTEMA ESPECIALISTA PARA DIAGNÓSTICO E
RECOMENDAÇÃO DE AÇÕES**

MACEIÓ - AL

2026

UNIVERSIDADE FEDERAL DE ALAGOAS

CAMPUS A.C. SIMÕES

ENGENHARIA DE COMPUTAÇÃO

DAVI CELESTINO DOS SANTOS BARBOSA, 23210541

JOÃO VICTOR TENÓRIO VENANCIO, 23211191

OTAVIO JOSHUA COSTA BRANDAO MENEZES, 23211189

LARISSA FERREIRA DIAS DE SOUZA, 23211102

**RELATÓRIO TÉCNICO: SISTEMA ESPECIALISTA PARA DIAGNÓSTICO E
RECOMENDAÇÃO DE AÇÕES**

Este relatório técnico descreve a especificação, arquitetura e implementação de uma ferramenta genérica voltada para a construção de sistemas especialistas baseados em conhecimento, aplicada a tarefas de diagnóstico e recomendação de ações ou tratamentos.

Orientador (a): Prof. Dr. Evandro de Barros Costa

MACEIÓ - AL

2026

SUMÁRIO

1. ARQUITETURA IMPLEMENTADA.....	4
2. ESTRUTURA DE REPRESENTAÇÃO DO CONHECIMENTO.....	4
Regra de Produção (Rule).....	4
Base de Conhecimento (KnowledgeBase).....	5
3. ESTRATÉGIA DE INFERÊNCIA UTILIZADA.....	5
A. Encadeamento para Frente (Forward Chaining).....	5
B. Encadeamento para Trás (Backward Chaining).....	6
C. Estratégia Híbrida.....	6
Propagação de Incerteza.....	6
4. IMPLEMENTAÇÃO DO MECANISMO DE EXPLICAÇÃO.....	6
5. EXEMPLOS DE CONSULTAS REALIZADAS.....	7
Exemplo 1: Encadeamento para Trás (Diagnóstico Específico).....	7
Exemplo 2: Encadeamento para Frente com Recomendação de Ação.....	7
6. LIMITAÇÕES E POSSÍVEIS MELHORIAS.....	8
Limitações.....	8
Possíveis Melhorias.....	8

1. ARQUITETURA IMPLEMENTADA

O sistema foi desenvolvido utilizando o paradigma de Programação Orientada a Objetos em Python, adotando uma arquitetura modular que isola completamente o conhecimento do domínio das estratégias de raciocínio lógico e da interface do usuário. Essa separação garante o caráter genérico da ferramenta, permitindo a reutilização do motor simbólico em múltiplos domínios (automotivo, médico, financeiro, etc.) sem qualquer necessidade de alteração no código-fonte.

A arquitetura do agente baseado em conhecimento é composta por quatro módulos principais:

- Camada de Dados e Persistência (KnowledgeBase): Centraliza o armazenamento de fatos (iniciais e inferidos) e regras de produção, isolando o estado da sessão atual das estruturas permanentes do conhecimento.
- Motor de Raciocínio (InferenceEngine): Centraliza os algoritmos de busca e inferência lógica, sendo responsável por coordenar a ativação de regras e a árvore de chamadas.
- Mecanismo de Explicação (ExplanationEngine): Analisa retrospectivamente a trilha de execução gerada pelo motor para traduzir o raciocínio simbólico em explicações inteligíveis para o usuário humano.
- Interface com o Usuário e Editor (KnowledgeBaseEditor e shell.py): Provê uma interface de linha de comando (CLI) interativa com assistentes dinâmicos de captura de dados e edição em tempo real da base.

2. ESTRUTURA DE REPRESENTAÇÃO DO CONHECIMENTO

O conhecimento é representado por meio de regras de produção baseadas na lógica de cláusulas de Horn simplificadas (estruturas declarativas SE <condições> ENTÃO <conclusão>) e tratamento de incerteza por Fatores de Certeza.

Regra de Produção (Rule)

Cada regra é modelada como uma estrutura de dados encapsulada que contém:

- id: Identificador único da regra (ex: "R01").

- conditions: Uma lista de strings representando os antecedentes (fatos que operam estritamente sob a conjunção lógica E).
- conclusion: O consequente (fato ou diagnóstico que será validado caso as condições sejam satisfeitas).
- description: Metadados textuais em linguagem natural para exibição em relatórios e explicações.
- certainty: Um valor em ponto flutuante no intervalo $[0..1]$ que quantifica o grau de confiança daquela regra específica.

Base de Conhecimento (KnowledgeBase)

A base de conhecimento divide dinamicamente a memória do sistema em:

- initial_facts: Fatos primitivos informados pelo usuário antes ou durante a execução de uma consulta.
- inferred_facts: Fatos derivados e injetados pelo motor de inferência, mapeados diretamente para o seu respectivo fator de certeza calculado.
- hypotheses: Lista de metas finais do sistema (diagnósticos terminais ou recomendações de ação).

A persistência do conhecimento é realizada de maneira transparente através da serialização e desserialização de arquivos estruturados em formato JSON, permitindo que especialistas externos criem bases complexas utilizando editores de texto convencionais.

3. ESTRATÉGIA DE INFERÊNCIA UTILIZADA

O InferenceEngine provê três estratégias distintas de busca e inferência sobre o grafo de conhecimento, permitindo ao especialista selecionar a melhor abordagem para o problema:

A. Encadeamento para Frente (Forward Chaining)

Uma abordagem direcionada pelos dados (data-driven). O algoritmo varre a base de regras continuamente enquanto novos fatos puderem ser adicionados à memória de trabalho. Em cada iteração, as regras cujas condições estão completamente satisfeitas na base de fatos são disparadas. O processo encerra quando ocorre uma iteração sem novos disparos (ponto fixo).

B. Encadeamento para Trás (Backward Chaining)

Uma abordagem direcionada por metas (goal-driven). Dado um objetivo (goal), o motor busca recursivamente na base por regras que possuam esse objetivo como conclusão. Para cada regra encontrada, suas condições tornam-se sub-objetivos que o motor tenta provar recursivamente. Caso um fato não possua regras capazes de inferi-lo, o motor aciona dinamicamente uma função de callback (ask_user) para interrogar o usuário sobre o sintoma observado.

C. Estratégia Híbrida

Uma abordagem combinada otimizada para o diagnóstico completo:

1. Fase de Expansão (Forward): O sistema varre os fatos iniciais fornecidos pelo usuário e propaga todas as conclusões imediatas possíveis, minimizando a quantidade de perguntas futuras.
2. Fase de Validação (Backward): O sistema itera sobre a lista de hipóteses e ações cadastradas e, para cada uma delas, executa o encadeamento para trás para validar sua viabilidade ou descartá-la de forma cirúrgica.

Propagação de Incerteza

A propagação dos fatores de certeza segue uma abordagem conservadora clássica baseada em lógica nebulosa/Fuzzy

4. IMPLEMENTAÇÃO DO MECANISMO DE EXPLICAÇÃO

O ExplanationEngine funciona rastreando e registrando eventos estruturados em tempo real durante as consultas através de uma lista de logs estruturados (trace). Cada entrada no histórico armazena o tipo de evento (ex: fired, try, confirmed), a regra associada e o fato em análise. Com base nessa trilha, o motor implementa duas funções de explicação fundamentais:

- Explicação "Por Quê?" (why): Utilizada quando o sistema interroga o usuário por um sintoma. Ela analisa as dependências lógicas imediatas e mostra ao usuário quais regras ativas necessitam daquele dado específico no momento e quais conclusões de nível superior dependem dele.
- Explicação "Como?" (how): Uma reconstrução recursiva pós-processamento da árvore de prova. Partindo de um diagnóstico ou ação concluída, o método

reconstrói o caminho de trás para frente, exibindo a regra que gerou a conclusão, as certezas associadas e a sub-prova detalhada de cada uma das pré-condições até alcançar os fatos primitivos informados pelo usuário.

5. EXEMPLOS DE CONSULTAS REALIZADAS

A eficácia do motor genérico foi validada utilizando uma base de conhecimento automotiva contendo 22 regras, 35 fatos e 6 hipóteses principais com recomendações.

Exemplo 1: Encadeamento para Trás (Diagnóstico Específico)

- Objetivo: Provar a hipótese `diagnostico_bateria_fraca`.
- Interações simuladas: * `motor_nao_gira` $\rightarrow$ Confirmado pelo usuário (True)
 - `luzes_fracas` $\rightarrow$ Confirmado pelo usuário (True)
 - `bateria_com_mais_de_3_anos` $\rightarrow$ Confirmado pelo usuário (True)
- Resultado do Sistema: Confirmado com 95% de certeza. A árvore gerada pelo comando `how` demonstrou com sucesso que `motor_nao_gira` e `luzes_fracas` dispararam a regra R01, inferindo o fato intermediário `tensao_bateria_baixa` com 90% de certeza, que por sua vez disparou a regra R02 junto à idade da bateria.

Exemplo 2: Encadeamento para Frente com Recomendação de Ação

- Fatos Iniciais: `temperatura_motor_alta = True`, `nivel_agua_baixo = True`, `vapor_sob_capo = True`.
- Saída do Motor: * Fatos Inferidos: ['`sistema_resfriamento_comprometido`', '`diagnostico_superaquecimento_motor`', '`acao_chamar_guincho_ou_mecanico`']
 - Diagnósticos/Ações: ['`diagnostico_superaquecimento_motor`', '`acao_chamar_guincho_ou_mecanico`']
- Análise do Mecanismo de Explicação: O motor demonstrou a capacidade de encadeamento profundo. Ele deduziu o comprometimento do sistema (Certeza: 90%), o que levou ao diagnóstico de superaquecimento (Certeza: 98%), que por consequência ativou automaticamente a regra de

recomendação final crítica (R25), indicando com 100% de certeza que o motorista deve desligar o veículo e solicitar assistência.

6. LIMITAÇÕES E POSSÍVEIS MELHORIAS

Apesar de atender de forma robusta e elegante a todos os requisitos arquiteturais e funcionais de um agente simbólico clássico, o sistema apresenta certas restrições técnicas inerentes ao escopo atual do projeto.

Limitações

1. **Monotonicidade Rígida:** O sistema assume que uma vez que um fato se torna verdadeiro, ele permanece verdadeiro até o fim da sessão. Não há suporte nativo para lógica não-monotônica (onde novos fatos poderiam invalidar conclusões prévias).
2. **Operador de Conjunção Restrito:** A sintaxe atual do interpretador de regras assume que todas as condições dentro de uma regra estão obrigatoriamente unidas por um operador E (AND). Condições alternativas exigem a criação de regras separadas com o mesmo consequente, o que expande artificialmente o tamanho do arquivo da base.
3. **Combinação Simplificada de Evidências:** Quando múltiplas regras independentes concluem o mesmo fato com intensidades diferentes, o sistema sobrescreve ou mantém o último valor processado, sem realizar a combinação cumulativa de certezas (fórmula de co-evidência).

Possíveis Melhorias

1. **Interface Gráfica Baseada na Web (GUI):** Substituir a interface em modo console (shell.py) por uma aplicação Web utilizando frameworks leves (como Streamlit ou Flask), tornando o questionário de sintomas visualmente intuitivo, com seletores e botões amigáveis para especialistas do domínio.
2. **Aprimoramento do Cálculo de Incerteza (MYCIN Completo):** Implementar o algoritmo clássico de combinação cumulativa de fatores de certeza para regras concorrentes que apontam para o mesmo diagnóstico.
3. **Suporte a Expressões Booleanas Complexas:** Refatorar a classe Rule e o parser de texto para aceitar operadores lógicos complexos de negação (NOT)

e disjunção (OR) agrupados por parênteses diretamente em uma única linha de condição, otimizando a legibilidade e manutenção da base de dados.